

Recall Does Not Move at a Trillion Rows

Routed search over compressed vectors, measured against the exact scan, on a shared research cluster

Andrew H. Bond
San José State University
San José, California, USA
andrew.bond@sjsu.edu

ABSTRACT

We report a measurement of routed inverted-file search over an index of one trillion 4-bit product-quantized vectors, built and queried on the National Research Platform’s shared Kubernetes cluster. The index is 500 servers of two billion rows each, 24.0 bytes per row, 24 TB in all, on one block volume per server. Against the exact full scan of the same compressed index, routed search returns recall at ten of 0.989 when it probes 32 of 2048 cells and 0.999 when it probes 128, on 100 queries. The same protocol gave 0.979 and 0.999 at one hundred million rows, 0.982 and 0.999 at ten billion, and 0.983 and 0.999 at one hundred billion. Recall against the full scan does not move across four orders of magnitude of corpus size. The measure is self-consistency of the compressed index on a synthetic corpus of intrinsic dimension 16, and the paper gives the float ground truth and real-embedding figures that bound it. The second contribution is the measurement design. Every job runs in one CPU and two gigabytes of memory, the smallest class the cluster offers, because the query set is read from a cache, at most two memory-mapped shards are open at a time, and scoring proceeds in blocks of 65536 rows. A pool of idempotent per-server jobs submitted through a polite controller, with exponential back-off and a distinction between a deferred submission and a lost one, carried 928 submissions to 849 completions over two days with nobody on duty. The reference scan cost 418 CPU-hours and the two routed passes 354. Every log, state file, partial result and the score are in the repository.

1 INTRODUCTION

Approximate nearest-neighbour search over compressed vectors is now routine at a billion rows [1–3]. The question this paper answers is narrower than most and, we think, worth answering on its own. When an inverted-file index is partitioned into shards that share one coarse quantizer and one rotation, and the corpus grows by adding shards rather than by densifying old ones, does the routing lose anything it would have found by scanning everything? The way to ask it is to build the whole index, scan the whole index exactly, and compare. We did that at one trillion rows.

The index was built and measured on the National Research Platform’s Nautilus cluster, a Kubernetes federation shared by many research groups across sites. A fleet measurement there is a matter of fair use by design. Jobs are small, independent and resumable, submissions are paced by a controller that yields when the cluster is busy, and nothing is reserved. The second half of the paper describes that design and what it did over two days.

The paper makes three claims and states one limit.

- Routed recall against the exact scan of the same index is 0.989 at 32 probes and 0.999 at 128 probes at 10^{12} rows, and the same protocol at 10^8 , 10^{10} and 10^{11} rows gave figures

within 0.01 of these at 32 probes and within 0.001 at 128 probes (Section 4).

- Every phase of the measurement runs in one CPU and two gigabytes of memory, given three choices in how a scan over memory-mapped shards is written (Section 5).
- A pool of idempotent jobs with a small number of explicit rules completes a two-day, 928-submission run on a shared cluster with nobody on duty, and every rule and every failure class is reported with its count (Section 6).
- The corpus is synthetic, of intrinsic dimension about 16, and the recall reported is against the compressed index’s own exact scan rather than against float ground truth. Section 7 gives the figures that bound the result, including a real-embedding pilot on which the routing holds and the compression alone does not.

2 THE INDEX AND THE CORPUS

Index. Each row is a 32-dimensional vector projected to 24 dimensions by one PCA basis shared by the whole fleet, quantized to 4 bits per dimension, and assigned to one of 2048 inverted-file cells by one shared coarse quantizer. A shard holds five million rows. A server holds 400 shards, two billion rows, on one block volume. The on-disk row is 12 bytes of codes, 8 bytes of per-row norms used by the asymmetric distance computation, and 4 bytes of cell membership, 24.0 bytes measured. Scoring is asymmetric distance computation over the codes, in row blocks, from memory-mapped shards. Because the basis and the coarse quantizer are shared, scores from different shards are comparable and a global top- k is the exact merge of per-shard top- k s. That property is what makes the measurement below exact rather than approximate.

Corpus. The corpus is defined by a seed and regenerated on demand, so the distributed build moves no corpus bytes. Each global shard draws a fresh rank-16 Gaussian basis, projects Gaussian coefficients through it with a linear decay of singular values from 1.0 to 0.3, and adds five percent noise, in an ambient dimension of 32. Its intrinsic dimension measures 15.4 to 17.0 at every size we tested and does not move with row count. Real embeddings such as Cohere Embed v3 at 1024 dimensions measure 52 to 61 by the same estimator. Hubness, the concentration of nearest-neighbour lists on a few points, does rise with corpus size on this recipe, at about 0.42 per decade at $k = 10$ against roughly 0.25 per decade on real data, because random 16-dimensional subspaces of a 32-dimensional space overlap. The corpus therefore gets harder in one respect as it grows and the recall reported below holds despite that.

Table 1: Recall at ten of routed search against the exact full scan of the same compressed index, by corpus size. Queries are 100 at 10^8 , 10^9 and 10^{12} rows and 500 at 10^{10} and 10^{11} . Reference wall time is the median per-server exact scan. The 10^{12} jobs ran on one CPU each where the earlier runs had six, so that column is not comparable across rows (Section 4.2).

rows	servers	32 probes	128 probes	ref. scan (s)
10^8	4	0.979	0.999	47
10^9	4	see text		589
10^{10}	8	0.982	0.999	8495
10^{11}	50	0.983	0.999	19041
10^{12}	500	0.989	0.999	2664

Scale points. The same protocol has been run at 10^8 rows on 4 servers, 10^9 on 4, 10^{10} on 8, 10^{11} on 50 and now 10^{12} on 500. Per-server work is constant from 10^{11} on, two billion rows, so the step from 50 to 500 servers changes nothing per node and changes only the number of chances to draw a slow or failing one.

3 PROTOCOL

The measurement has four phases, each a pool of independent jobs on the cluster’s batch system, every submission through a controller that paces submissions, caps concurrency at twenty and yields while the cluster is busy.

- (1) **Query cache.** One job derives the query set, the first 25 rows of each of four seeded shards, 100 queries, and writes it to the shared volume. The queries are corpus rows by construction, so every server sees the same queries without regenerating anything.
- (2) **Reference.** One job per server scans that server’s two billion rows exactly with the asymmetric distance computation and writes its top-10 per query with scores.
- (3) **Routed.** One job per server runs the inverted-file search at 32 and at 128 probes and writes the same shape of partial.
- (4) **Score.** One job merges the 500 reference partials into the exact global top-10, merges the routed partials the same way, and reports recall at ten of the routed set against the reference set, per probe width, averaged over queries.

Every job is idempotent on the volume. A job whose output already exists prints that fact and exits, and outputs are written to a temporary name and renamed, so a job interrupted mid-write leaves nothing behind. This is the property every recovery rule in Section 6 relies on.

The score program has not changed since the 10^9 run. The two phase scripts changed at this run in the ways Section 5 describes, none of which touch what is computed.

4 RESULTS

4.1 Recall across four orders of magnitude

Table 1 and Figure 1 give the number the run exists to produce. At 10^{12} rows, routed search at 128 probes of 2048 cells returns 0.999 of the exact scan’s top ten, and at 32 probes 0.989. At 128 probes the figure is 0.999, 0.9988 and 0.9986 at the three smaller scales, a spread of 0.0004. At 32 probes it rises slowly with scale, 0.979, 0.982,

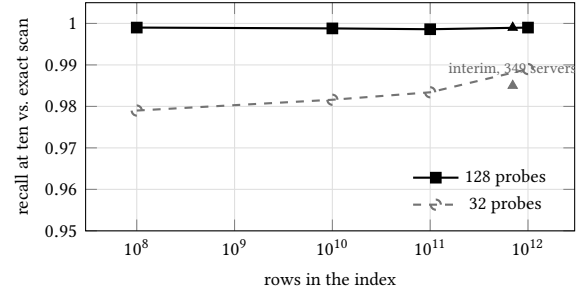


Figure 1: Recall against the exact scan by corpus size. The triangles are the interim score taken while the 10^{12} run was two thirds through its routed phase, on the 349 servers that had finished both phases, 698 billion rows. They sit on the final points.

Table 2: Per-server wall time in the 10^{12} run, one CPU per job, 500 servers per phase, and the CPU-hours each phase consumed.

phase	median	mean	p90	max	CPU-h
exact scan	2664 s	3008 s	4013 s	16308 s	418
routed, 32	1064 s	1131 s		14286 s	157
routed, 128	1335 s	1420 s		14524 s	197

0.983, 0.989, which is consistent with the merge over more servers filling the tail of the shortlist and is not something we claim to explain. The interim score, taken on the first 349 servers to finish both phases, gave 0.985 and 0.999, and recall at 128 probes on the first half of those 349 servers alone was also 0.999. The number is stable inside the run as well as across runs.

The 10^9 row has no routed-versus-scan figure in the table because that run’s purpose was different. It carried a cold store of the original float vectors, so it measured true recall against float ground truth. The routed shortlist at 128 probes returned 0.999 of the compressed scan there too, the compressed scan itself found only 0.592 of the true neighbours, and reranking the shortlist against the stored originals raised that to 0.991. That row is the reason Section 7 exists.

4.2 Cost

Table 2 gives what the measurement cost. The exact scan of two billion rows at one CPU takes 44 minutes at the median and the routed pass 18 to 22 minutes. The ratio between the routed and exact rows is not a latency figure for the method. These jobs are read-bound on network block storage, spend a fixed minute or two on image pull, dependency install and clone, and open 400 shards before scoring. The 10^{11} run, on six CPUs per job, reported 14.7 and 8.5 as the same ratio, and neither number is the method’s. The tails are node variance. The 16308 s maximum is one server on one node that also produced the slowest routed job, and the p90 is within 1.5 of the median.

Table 3: The pool over two days, 2026-09-25 18:01 UTC to 2026-09-27 08:59 UTC. Reference phase counts exclude 152 servers scanned earlier.

submissions	928
completions (348 scan, 500 routed, 1 score)	849
recycles after a failed job	75
of which a storage read fault on one node	24
node lost during the job	13
node unreachable, pod stuck	13
container start failure	2
recycles after 45 minutes pending	26
submissions deferred by the controller	1
servers parked or given up	0

5 ONE CPU AND TWO GIGABYTES PER JOB

Every job in the measurement requests one CPU and two gigabytes of memory. That is the smallest class the cluster offers, it is the class in which a fleet of hundreds of small jobs takes the least from other users, and the sharded index needs nothing more once three choices are made in how a scan over memory-mapped shards is written.

- (1) **Queries from the cache.** Deriving the 100 queries from the seed means regenerating four whole five-million-row blocks, about 640 MB each plus temporaries. The query-cache phase writes the set once and every scan reads the file.
- (2) **Two open shards.** A memory-mapped shard still materializes its row-id and tombstone arrays, 45 MB per five-million-row shard, and the sharded index holds 128 shards open by default, 5.8 GB across a full scan. The scan holds two shards open and the routed pass eight, with two worker threads.
- (3) **Blocks of 65536 rows.** The scoring block bounds the per-block temporaries to tens of megabytes.

A pilot scan at this shape finished in 1912 s with CPU averaging 884 millicores and peaking at 979, memory averaging 1242 MiB and peaking at 1469, and a process resident set of 346 MB at the end. A pilot routed pass finished in 36 minutes at 475 millicores average, memory averaging 1258 MiB and peaking at 1568, resident set 165 MB. The memory above the resident set is clean page cache from the mapped index, which the kernel reclaims under the limit. The routed pod was observed at the two-gigabyte limit with 224 MB anonymous and 1.9 GB file-backed and completed normally. The score job, which merges 1500 small partials, runs in the same class.

6 RUNNING A POOL ON A SHARED CLUSTER

The measurement was driven by a pool of at most twenty jobs in flight with no wave barrier, checkpointed per phase, restartable, and able to adopt jobs already on the cluster after a restart. Table 3 gives what it did over two days. The rules come first because the counts are only meaningful against them.

Rules. A job whose Kubernetes object reports success is done. A job whose object reports failure is recycled, meaning its object is deleted and, after an exponential back-off from two minutes to

thirty, resubmitted, up to forty times before the server is parked and swept again later. A job whose pod has not reached the running state 45 minutes after submission is recycled the same way. A job submitted through the controller but not yet visible as an object is waited on for ninety polls, one a minute, before it is treated as lost. A job running longer than sixteen hours is treated as wedged.

The deferral rule. The submission controller defers creating a job while the namespace already has several pending or the cluster is busy, and retries for up to about an hour. A driver that treats a deferred submission as a lost one resubmits, and every resubmission queues a duplicate behind the original it was meant to replace. Ninety polls of patience separates the two cases. The run needed the rule once, for the score job.

Failure classes. Two dominated, and both are the ordinary weather of a shared cluster. One node returned a storage read fault to every job placed on it within 3 to 13 minutes of start, after its block volume had attached. Retries landed elsewhere and finished, at a cost of about 130 slot-minutes out of 9600 over the first eight hours. The second class, pods pending past 45 minutes, was benign in every instance checked, because the original pod had run and the retry found its output within minutes. That is what idempotent jobs buy.

Pods that stop answering. Three scan pods stayed in a terminating state on nodes that had become unreachable, and one routed pod ran at 34 millicores for 97 minutes on a node with a slow storage path. The sixteen-hour wedge rule covers both cases eventually. Removing the four pods by hand let the job controller replace them, and the replacements finished within an hour. A rule for pods terminating longer than thirty minutes belongs in the driver, and the count of manual actions over the two days is those four removals and one resubmission of the score job.

Provenance. The job containers cloned the repository’s default branch at run time, and twelve commits appear in pod logs across the run. The two phase scripts came from a configuration map fixed for the run, and the merge and score code did not change. Pinning the clone to a commit is the one change to the descriptor for the next run.

7 WHAT THE RESULT DOES AND DOES NOT SHOW

Recall of the routed shortlist against the exact scan of the same compressed index is flat from 10^8 to 10^{12} rows. That is a statement about the inverted-file layer, its shared coarse quantizer and its merge. The routing loses nothing the exact scan finds, at either probe width, when the corpus grows by adding shards.

It is not a statement about recall against the truth. The compressed scan at 10^9 rows found 0.592 of the float neighbours, and on a real corpus of 15 million Cohere Embed v3 vectors the same index found 0.693 of them by exact scan, 0.689 by routing at 256 probes, and 0.981 after reranking the routed shortlist against stored originals. On that real corpus the routed shortlist returned 0.990 of the compressed scan at 256 probes, so the routing result transfers, and the compression result does not. Four bits per dimension at this rate loses neighbours that only a rerank tier recovers, and the

trillion-row corpus, at intrinsic dimension 16 against a real embedding’s 52 to 61, cannot show that. It is an easy corpus for the code and a hard one only for the hubness the shards’ overlap induces.

Two further limits. One hundred queries is few. The exact scan is compute-bound in the query count and 500 queries would have cost about five times the 418 CPU-hours. And a cold store of originals at 10^{12} rows would be 128 TB of float vectors, so true recall and reranking remain established at 10^9 and at the 15-million-row real pilot, not here.

8 RELATED WORK

Product quantization with asymmetric distance computation and inverted files is the basis of the index [2], and locally optimized codebooks per cell [4] the nearest relative of the shared-basis arrangement we use to keep scores comparable across shards. Faiss [3] established billion-scale practice on one machine, and SPANN [1] the disk-resident inverted index at a billion rows. Graph indexes [5] are the usual alternative and are not what this paper measures. Hubness [6] is the property of the corpus that does grow with scale here. This paper adds a measurement at 10^{12} rows against the exact scan of the same index and the design that obtained it on a shared cluster.

9 CONCLUSION

A trillion-row compressed index was built and measured exactly on a shared research cluster in the smallest resource class the platform offers, by jobs that read their queries from a cache, hold two shards

open and score in blocks, submitted through a controller that yields when the cluster is busy. On that index, routing loses nothing the exact scan finds, at 10^8 rows or at 10^{12} . The run cost 772 CPU-hours and two days, and it is reproducible from the repository, which carries the seeds, the scripts, the driver, the state files, the driver logs, the pilots, the interim score and the final score with every per-server wall time.

ACKNOWLEDGMENTS

This work used the National Research Platform’s Nautilus cluster, supported by the National Science Foundation. We thank its operators.

REFERENCES

- [1] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighbor Search. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.
- [4] Yannis Kalantidis and Yannis Avrithis. 2014. Locally Optimized Product Quantization for Approximate Nearest Neighbor Search. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2321–2328.
- [5] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.
- [6] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. 2010. Hubs in Space: Popular Nearest Neighbors in High-Dimensional Data. *Journal of Machine Learning Research* 11 (2010), 2487–2531.